③ The launch template used as a blueprint for spinning up EC2 instances

④ Looks up the AMI with the Imagery server preinstalled from the region map (see listing 16.9)

⑤ Launches virtual machines of type t2.micro to run examples under the Free Tier

⑥ Configures a network interface (ENI) with a public IP address and the security group of the server

⑦ Each virtual machine will execute this script at the end of the boot process. The script starts the Node.js server.

⑧ Creates an Auto Scaling group that manages the virtual machines running the Imagery server

⑨ References the launch template

⑩ The Auto Scaling group will spin up at least one and no more than two EC2 instances.

The Auto Scaling group will register and deregister virtual machines at the target group.

The Auto Scaling group will replace EC2 instances that fail the target group's health check.

Spins up EC2 instances distributed among two subnets and, therefore, two AZs

That's it for the server. Next, you need to deploy the worker.

Deploying the worker with an Auto Scaling group

Deploying the worker works similar to the process for the server. Instead of a load balancer, however, the queue is used for decoupling. Please note that we already explained how to create a SQS in listing 16.8. Therefore, all that's left is the Auto Scaling group and a launch template. The next listing shows the details.

Listing 16.11 Load balancer and Auto Scaling group for the Imagery worker

```
WorkerLaunchTemplate:                                    ①
  Type: 'AWS::EC2::LaunchTemplate'
  Properties:
    LaunchTemplateData:
      IamInstanceProfile:                                ②
```